departamento de eletrónica, telecomunicações e informática

Curso: 8295 – Licenciatura em Engenharia Informática
Disciplina: 40381 – Sistemas Operativos
Ano Letivo: 2026/2027

Relatório

Implementação de um comando de verificação de modificações em ficheiros de um sistema (Vigia-Pirata – vp)

Autores:
132982 João Pedro Alexandre Reis
133155 Rúben da Silva Marques

Turma: P2
Grupo: 3

Data: 06/11/2026
Docente: Pedro Azevedo Fernandes

Resumo:

O trabalho consiste no desenvolvimento de um comando em Bash, denominado Vigia-Pirata (vp), que permite monitorizar a integridade dos ficheiros de um ou mais diretórios. Através de uma base de dados, são guardadas informações sobre os ficheiros, incluindo os seus metadados o respetivo hash SHA-256. O programa permite detetar ficheiros novos, apagados ou modificados, identificando alterações na posse, tamanho, conteúdo, permissões e data de modificação.

Conteúdo

Relatório 1
Introdução 3
Descrição do problema 3
Procedimento 4
Resultados 11
Análise dos Resultados 12
Conclusões 13
Referências 13

Introdução

Este relatório descreve o desenvolvimento do Vigia-Pirata (`vp`), um comando escrito em Bash no âmbito da unidade curricular de Sistemas Operativos. O `vp` permite monitorizar a integridade dos ficheiros de um ou mais diretórios: num primeiro momento regista o estado dos ficheiros numa base de dados e, mais tarde, compara esse registo com o estado atual do sistema, assinalando o que foi modificado, apagado ou adicionado.

Este tipo de verificação é útil para detetar alterações indevidas em ficheiros importantes, como a modificação do conteúdo de um ficheiro ou a mudança das suas permissões. Para detetar alterações de conteúdo sem guardar uma cópia dos ficheiros, o `vp` guarda o hash SHA-256 de cada um, em conjunto com os seus metadados.

O trabalho foi desenvolvido de forma incremental: começámos pela leitura e validação dos argumentos, seguiram-se o modo de proteção, o modo de verificação e o sistema de alertas. Os testes em cenários reais levaram a vários ajustes no código do `vp`.

Descrição do problema

O objetivo é implementar em Bash o comando vp, com a seguinte sintaxe:

```
./vp.sh [{-p <dir1[<dir2>...] | -v}] [-f <ficheiro>] [-e <ignore_pattern>] [-s]
```

Opção	Função
`-p`	Modo de proteção: guarda informação sobre os ficheiros de um ou mais diretórios (incluindo subdiretórios).
`-v`	Modo de verificação: deteta alterações aos ficheiros desde que foram protegidos.
`-f`	Nome do ficheiro onde a base de dados é guardada (por defeito `/tmp/.pdb`).
`-e`	Padrão de ficheiros a excluir da proteção e da verificação (por exemplo `*.log`).
`-s`	Envia os alertas, com prioridade alta, para o sistema de logging através do `logger`.

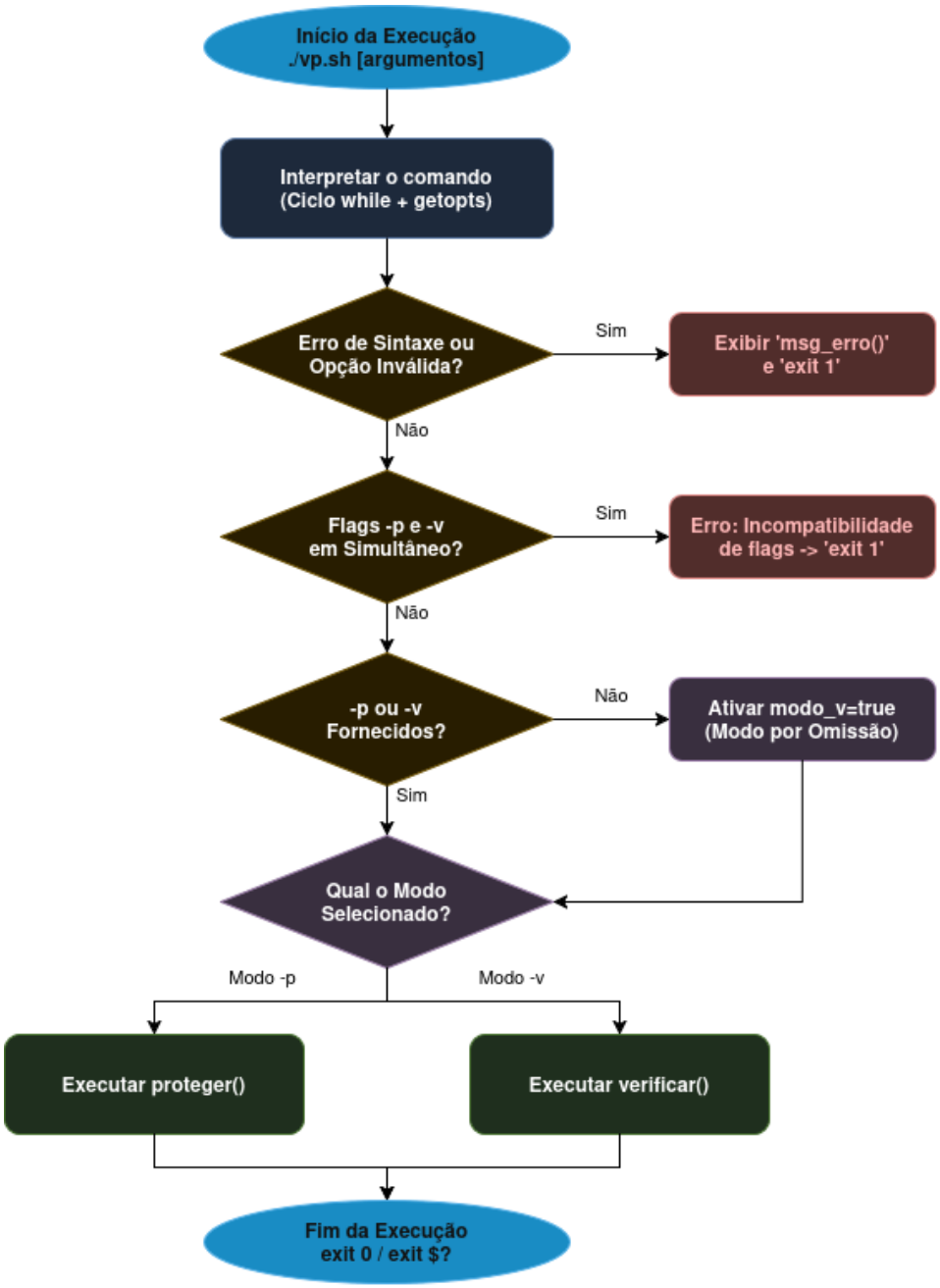
Para cada ficheiro regular, o modo de proteção guarda o dono (`USER`), o grupo (`GROUP`), o tamanho (`SIZE`), as permissões em octal (`PERM`), a data de modificação (`MTIME`) e o hash SHA-256. O modo de verificação compara esta informação com o estado atual e classifica cada ficheiro como modificado, apagado ou novo, indicando se a alteração foi de posse, de permissões, de conteúdo ou de data de modificação.

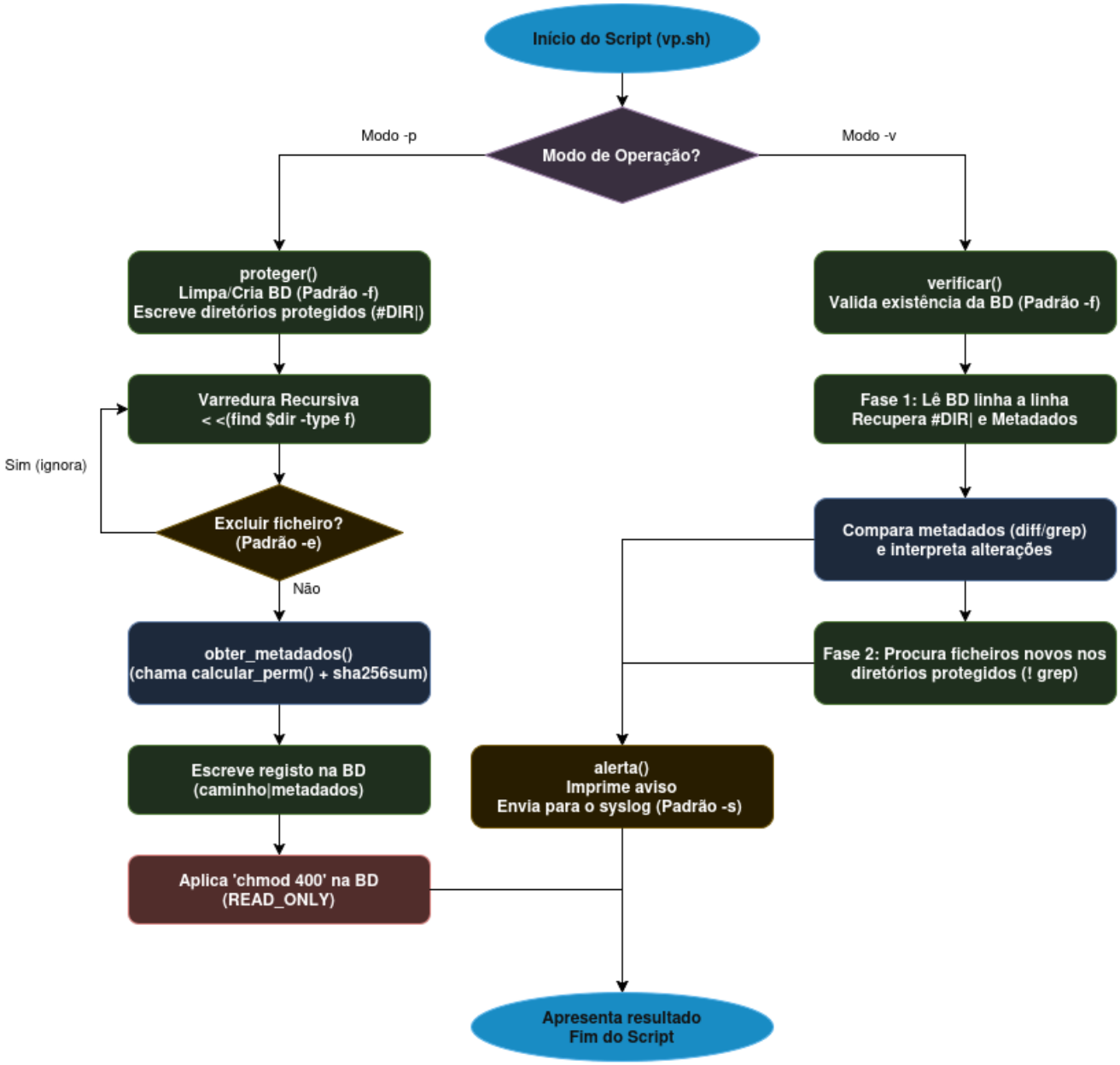
As opções `-p` e `-v` não podem ser usadas em simultâneo e, se nenhuma for indicada, o programa assume `-v` por defeito.

Procedimento

O script `vp.sh` foi organizado em funções, desenvolvidas pela ordem apresentada de seguida. As figuras mostram excertos do código final.

Para facilitar a compreensão do funcionamento do `vp`, elaborámos dois fluxogramas. O primeiro mostra o que acontece quando o comando é executado: a leitura e validação das opções, o tratamento dos erros e a escolha do modo de operação. O segundo mostra o funcionamento interno de cada modo, proteção e verificação, e o papel das funções auxiliares.





1. Leitura e validação dos argumentos

Para analisar as opções usámos o `getopts`, recomendado no guião, porque distingue opções booleanas de opções com argumento (guardado em `$OPTARG`) e permite agrupar opções (por exemplo `-vf`). Com o prefixo `:` na lista de opções (`":pvf:e:s"`), os erros de opção inválida e de argumento em falta ficam a cargo do próprio script: em ambos os casos é mostrada uma mensagem e o modo de uso, e o programa termina com `exit 1`.

A opção `-p` aceita mais do que um diretório, mas o `getopts` pára no primeiro argumento que não é uma opção. Por isso colocámo-lo dentro de um ciclo `while` exterior: quando o `getopts` pára, as opções já lidas são retiradas da linha de comandos com `shift $((OPTIND - 1))`, o argumento seguinte é guardado no array `dir` e o `OPTIND` volta a `1` para que o `getopts` retome a procura de opções. Os valores lidos ficam em variáveis (`modo_p`, `modo_v`, `ficheiro_bd`, `padrao_ignorar`, `usar_logger` e `dir`) usadas depois pelas funções principais. No fim, se forem pedidos `-p` e `-v` em simultâneo o programa termina com erro, e se não for pedido nenhum é assumido `-v`.

```bash
# executa enquanto existirem argumentos na linha de comandos
while [ $# -gt 0 ]; do

    # o getopts processa todas as flags seguidas até encontrar um argumento sem "-"
    while getopts ":pvf:e:s" opt; do
        case "$opt" in
            p) modo_p=true ;;
            v) modo_v=true ;;
            f) ficheiro_bd="$OPTARG" ;;
            e) padrao_ignorar="$OPTARG" ;;
            s) usar_logger=true ;;
            \?) echo "Invalid switch -$OPTARG"; msg_erro; exit 1 ;;
            :)  echo "Falta argumento para -$OPTARG"; msg_erro; exit 1 ;;
        esac
    done

    # elimina da linha de comandos as flags que o getopts acabou de ler
    shift $((OPTIND - 1))

    # se ainda sobrar algum argumento, significa que o getopts parou num diretório (argumento de -p)
    if [ $# -gt 0 ]; then
        dir+=("$1") # guarda o diretório no array (argumento de -p)
        shift               # avança 1 argumento para "consumir" o diretório (argumento de -p)
        OPTIND=1            # reinicia o índice do getopts para ele voltar a procurar flags
    fi
done
```

2. Recolha de metadados

A função `obter_metadados()` devolve numa linha o dono, o grupo, o tamanho, as permissões, a data de modificação e o hash do ficheiro. Como não era possível usar o `stat`, os valores são extraídos do `ls -l` com o `awk` (colunas 3, 4, 5 e 1). A data de modificação é obtida com o comando `ls -l --time-style=+%Y-%m-%d_%H:%M:%S`, que dá a data num formato fixo e sem espaços, ficando toda na coluna 6. O hash vem do `sha256sum`.

O `ls -l` devolve as permissões em texto (por exemplo `-rw-r--r--`). A função auxiliar `calcular_perm()` ignora o primeiro carácter (tipo de ficheiro) e analisa os restantes nove em três blocos (dono, grupo e outros), somando 4 para `r`, 2 para `w` e 1 para `x`. O resultado é a notação octal habitual, por exemplo `644` ou `755`.

```bash
function calcular_perm(){ #r=4;w=2;x=1;-=0
    local p="${1:1:9}" # ${variavel:inicio:tamanho}, a posição 0 não interessa pois apenas indica o tipo de ficheiro
    local numero="" i grupo valor #cria 4 variaveis, numero que é o resultado inicializado com ""
                                  #i é a posição onde começa cada grupo, grupo sao as 3 letras e o valor é o valor do grupo (rwx)

    for i in 0 3 6; do #ciclo acontece 3 vezes
        grupo=${p:$i:3}
        valor=0

        if [ "${grupo:0:1}" != "-" ]; then #permissao tem a ordem rwx
            valor=$((valor + 4))
        fi
        if [ "${grupo:1:1}" != "-" ]; then
            valor=$((valor + 2))
        fi
        if [ "${grupo:2:1}" != "-" ]; then
            valor=$((valor + 1))
        fi

        numero="$numero$valor" #junta o textl de numero e valor. ex: ""+"7"
    done

    echo "$numero"
}
```

```bash
function obter_metadados() {
    local ficheiro="$1"

    #ls -l ficheiro vai imprimir uma coisa do genero -rw-r--r-- 1 jose jose 7 Oct 29 22:58 dados/a.txt
    local user=$(ls -l "$ficheiro" | awk '{print $3}') #awk imprime so a coluna 3
    local group=$(ls -l "$ficheiro" | awk '{print $4}')
    local size=$(ls -l "$ficheiro" | awk '{print $5}')
    local perm=$(calcular_perm "$(ls -l "$ficheiro" | awk '{print $1}')") # aqui é aproveitada a coluna 1 que depois é utilizada pela calcular_perm
    # data de modificação: o --time-style dá a data num formato fixo e sem espaços (ex: 2026-10-29_22:58:10), logo fica tudo na coluna 6
    local mtime=$(ls -l --time-style=+%Y-%m-%d_%H:%M:%S "$ficheiro" | awk '{print $6}')
    local sha256=$(sha256sum "$ficheiro" | awk '{print $1}') #imprime varios caracteres e se for alterado uma unica letra o conjunto de caracteres muda


    # Retorna tudo numa linha separado por |
    echo "USER=$user, GROUP=$group, SIZE=$size, PERM=$perm, MTIME=$mtime, SHA256=$sha256"
}
```

3. Exclusão de ficheiros (-e)

A lógica da opção `-e` é necessária tanto na proteção como na verificação, pelo que foi isolada na função `excluir()`, evitando código duplicado. A função recebe o caminho do ficheiro e compara-o, com um `case`, com o padrão `$padrao_ignorar`: devolve `0` se coincidir (o ficheiro deve ser ignorado) e `1` caso contrário. Se `-e` não for usada, devolve sempre `1`. O padrão é comparado com o caminho completo do ficheiro, pelo que funciona com padrões como `.log` ou `/tmp/`.

```bash
function excluir() {
    if [ -n "$padrao_ignorar" ]; then # se -e foi chamado
        case "$1" in
            $padrao_ignorar) return 0 ;; # 0 = Verdadeiro (coincide com o padrão, é para excluir)
            *) return 1 ;;               # 1 = Falso (não coincide)
        esac
    fi
    return 1;
}
```

4. Modo de proteção

A função `proteger()` cria a base de dados de proteção, pela seguinte ordem:

1. Verifica se foi indicado pelo menos um diretório; caso contrário mostra erro e termina.
2. Remove a base de dados anterior (`rm -f`) e cria uma nova, vazia, em `/tmp/.pdb` ou no ficheiro indicado em `-f`. A remoção prévia é necessária porque, no fim, a base de dados fica só de leitura.
3. Escreve uma linha `#DIR|diretorio` para cada diretório indicado que exista, registando assim os diretórios protegidos.
4. Percorre cada diretório com `find "$diretorio" -type f`, o que inclui os subdiretórios. Os ficheiros que coincidem com o padrão de `-e` são ignorados e, para os restantes, escreve-se uma linha `caminho|metadados`.
5. Aplica `chmod 400` à base de dados e mostra o número de ficheiros processados e os diretórios protegidos.

```bash
local diretorios_protegidos=()

    for diretorio in "${dir[@]}"; do
        if [ -d "$diretorio" ]; then
            echo "#DIR|$diretorio" >> "$ficheiro_bd" # guarda na BD os nomes dos diretorios protegidos (linhas #DIR|nome)
            diretorios_protegidos+=("$diretorio")
        else
            echo "O diretório '$diretorio' não existe."
        fi
    done
```

```bash
    while IFS= read -r ficheiro; do # IFS evita que o read apague espaços no inicio/fim da linha e -r evita que \ seja interpretado como caracter especial

            if excluir "$ficheiro"; then
                continue # se for para excluir salta para o outro ficheiro
            fi

            metadados=$(obter_metadados "$ficheiro") # guarda as informações em metadados

            if [ $? -eq 0 ]; then # se o ultimo comando returnou 0 ou seja foi possivel obter metadados fazemos:
                echo "$ficheiro|$metadados" >> "$ficheiro_bd" #escreve o caminho e os metadados na bd
                ficheiros=$((ficheiros + 1))
            else # se nao foi possivel obter os metadados return 1 e temos erro
                echo "Não é possível obter_metadados de '$ficheiro'."
            fi

    done < <(find "$diretorio" -type f)
done

chmod 400 "$ficheiro_bd" # deixa a bd so com leitura para o dono ("acesso READ_OLNY")
```

O resultado é uma base de dados com o seguinte formato:

```
#DIR|<diretorio>
#DIR|<diretorio>
⋮
<caminho do ficheiro>|USER=<dono>, GROUP=<grupo>, SIZE=<bytes>, PERM=<octal>,
MTIME=<data>, SHA256=<hash>
⋮
```

5. Modo de verificação

A função `verificar()` termina com erro se a base de dados não existir e está dividida em duas partes. Na primeira lê-se a base de dados linha a linha, separando cada linha em caminho e metadados guardados. As linhas `#DIR|` preenchem o array `dirs_protegidos`. Para cada ficheiro registado que não seja excluído por `-e`, confirma-se se ainda existe: se não existir, é assinalado como apagado. Se existir, calculam-se os metadados atuais e comparam-se com os guardados.

```
while IFS="|" read -r ficheiro guardado; do  # IFS diz ao read para dividir linha quando encontrar |
                                             #divide em dois a 1 parte fica em ficheiro a 2 em guardado


        # linhas "#DIR|nome" guardam os diretorios protegidos
        if [ "$ficheiro" = "#DIR" ]; then
            dirs_protegidos+=("$guardado")
            continue
        fi


        if excluir "$ficheiro"; then #se o ficheiro corresponder ao padrao do -e é ignorado
            continue
        fi

        verificados=$((verificados+1))

        if [ ! -f "$ficheiro" ]; then # se -f dizer que ficheiro não existe
            alerta "$ficheiro foi APAGADO" #imprime mensagem e envia para o log se houver -s
            echo " Informação inicial: $guardado"
            echo " Informação atual: [Ficheiro não encontrado no sistema]"
            echo "$sep"
            apagados=$((apagados + 1))
            continue # salta o resto da volta
        fi
```

Quando há diferenças, o `diff` é aplicado aos metadados, com um campo por linha (usando `tr`), e o `grep` identifica o que mudou: `USER` ou `GROUP` correspondem a alteração de Posse, `PERM` a Permissões, `SIZE` ou `SHA256` a Conteúdo e `MTIME` a Data de modificação. As categorias detetadas são juntas numa descrição (por exemplo "Conteúdo e Data de modificação"), o que permite detetar um touch mesmo que o tamanho e o hash se mantenham.

```
atual=$(obter_metadados "$ficheiro") # calcula os metadados de agora do ficheiro

 if [ "$atual" != "$guardado" ]; then # se forem diferentes isto vai acontecer
     # um campo por linha, para o diff mostrar só os que mudaram
     dif=$(diff <(tr ',' '\n' <<< "$guardado") <(tr ',' '\n' <<< "$atual"))

     desc=""
     grep -qE "USER|GROUP" <<< "$dif" && desc="$desc Posse"
     grep -q "PERM" <<< "$dif" && desc="$desc Permissões"
     grep -qE "SIZE|SHA256" <<< "$dif" && desc="$desc Conteúdo"
     grep -q "MTIME" <<< "$dif" && desc="$desc Data_de_modificação" # com _ para não ser partido pelo "e" abaixo
     desc=${desc# }
     desc=${desc// / e }
     desc=${desc//_/ } # volta a pôr espaços em "Data de modificação"
```

A segunda parte deteta os ficheiros novos. Percorrem-se, com `find`, os diretórios registados nas linhas `#DIR|` e, para cada ficheiro não excluído por `-e`, procura-se na base de dados o caminho seguido de `|` com `grep -qF`. Se não for encontrado, o ficheiro é assinalado como novo.

```
for d in "${dirs_protegidos[@]}"; do # percorre os elementos do array
        while IFS= read -r ficheiro; do
            if excluir "$ficheiro"; then
                continue
            fi # salta os excluidos

            if ! grep -qF "$ficheiro|" "$ficheiro_bd"; then # -q responde se foi enocntrado ou nao mas nao imprime nada e o -F diferfencia os caracteres especiais
               #se nao tiver na bd acontece ito // procura o caminho na bd
                alerta "$ficheiro é NOVO"
                echo "  Informação atual:  $(obter_metadados "$ficheiro")"
                echo "$sep"
                novos=$((novos + 1))
            fi
        done < <(find "$d" -type f) # procura dentro do diretorio d e os subdiretorios os ficheiros apenas file (f)
    done
```

No fim, a função mostra o resumo com o total de ficheiros verificados, modificados, apagados e novos. A última instrução devolve sucesso (0) apenas se não houve ficheiros modificados, apagados ou novos, e o script termina com esse valor.

6. Alertas

A função `alerta()` concentra a emissão das mensagens de alteração (MODIFICADO, APAGADO e NOVO). Mostra sempre o texto no terminal e, se a opção `-s` estiver ativa, envia também a mensagem para o registo do sistema com `logger -p user.alert -t vp`.

```
function alerta() {
    echo "$1"
    if [ "$usar_logger" = true ]; then
        logger -p user.alert -t vp "$1" # -p defininimos o nivel de prioridade o -t é para colcoar o programa vp a gerar o log
    fi
}
```

7. Ajustes após os testes

Ao testar o script em cenários reais, ajustámos o código para cumprir os requisitos do enunciado:

- **Data de modificação (`MTIME`):** passou a ser guardada em `obter_metadados()` e comparada em `verificar()`, permitindo detetar alterações feitas com `touch` mesmo quando o tamanho e o hash não mudam.
- **Recursividade (`find`):** o varrimento com `for` simples foi substituído por `find "$diretorio" -type f` com *process substitution* (`< <(find ...)`), que cobre as subpastas e evita que o `stdin` seja consumido pelos ciclos `while`.
- **Diretórios protegidos (`#DIR|`):** passaram a ser guardados na base de dados, de modo que a deteção de ficheiros novos seja feita exatamente nos diretórios e subdiretórios originalmente protegidos.
- **Base de dados só de leitura:** `chmod 400` no fim de proteger, com `rm -f` prévio para permitir repetir o modo `-p` sobre uma base de dados já existente.
- **Diretórios inexistentes**: adicionamos um `if` a função `proteger()` que não protege um diretório que não existe deixando o campo de "Diretórios Protegidos" vazio.

Resultados

Os testes foram automatizados no script `testes.sh`, que cria o ambiente com dois diretórios, executa o `vp.sh`, simula alterações em ambos e remove tudo no fim. Nas figuras seguintes os hashes aparecem abreviados e algumas linhas foram partidas para caberem na página.

Teste 1 – Modo de proteção com múltiplos diretórios

Estrutura inicial, em `/tmp/demo_vp1` e `/tmp/demo_vp2`:

```
/tmp/demo_vp1
├── app.log
├── ficheiro1.txt
├── ficheiro2.txt
└── documentos
      └── manual.txt
/tmp/demo_vp2
└── backup
      └── dados.txt
```

```
→ Comando executado: ./vp.sh -p /tmp/demo_vp1 /tmp/demo_vp2 -f /tmp/.pdb_demo -e "*.log"
Base de dados de proteção /tmp/.pdb_demo criada com sucesso.
Ficheiros processados: 4 | Diretórios protegidos: /tmp/demo_vp1 /tmp/demo_vp2

→ Conteúdo gerado na Base de Dados (/tmp/.pdb_demo):
#DIR|/tmp/demo_vp1
#DIR|/tmp/demo_vp2
/tmp/demo_vp1/ficheiro2.txt|USER=rubenmarques, GROUP=rubenmarques, SIZE=31, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=8b12e603872afe2543278d9d248834eb
fde21239366f31d347f122c99cee5266
/tmp/demo_vp1/ficheiro1.txt|USER=rubenmarques, GROUP=rubenmarques, SIZE=31, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=6e2d9af781d1156654bffbd1904a26e2
1f008a9ba348974f1ef14de6f1144257
/tmp/demo_vp1/documentos/manual.txt|USER=rubenmarques, GROUP=rubenmarques, SIZE=28, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=d85cf9342c24fdb57e41c218
5716be1b7c865f1e428c4c51aed6b804dcf43e54
/tmp/demo_vp2/backup/dados.txt|USER=rubenmarques, GROUP=rubenmarques, SIZE=30, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=160bcfe05cc4ab0baa741f0136dde
3792c09bdb73e6d5005798c8240f3ebcb1a
```

Teste 2 – Verificação após alterações

Antes da verificação foram feitas as seguintes alterações, nos dois diretórios:

```
→ Comandos de alteração executados:
   1. echo " Alterado!" >> /tmp/demo_vp1/ficheiro1.txt
   2. chmod 777 /tmp/demo_vp1/ficheiro2.txt
   3. rm /tmp/demo_vp1/documentos/manual.txt
   4. echo "Sou um ficheiro novo em DIR1" > /tmp/demo_vp1/ficheiro_novo.txt
   5. echo "Outro log" > /tmp/demo_vp1/novo.log
   6. echo " Alterado em DIR2!" >> /tmp/demo_vp2/backup/dados.txt
   7. echo "Novo ficheiro em DIR2" > /tmp/demo_vp2/backup/novo_dir2.txt

[PASSO 4] A executar o vp.sh em modo de VERIFICAÇÃO (-v)...
→ Comando executado: ./vp.sh -v -f /tmp/.pdb_demo -e "*.log" -s
/tmp/demo_vp1/ficheiro2.txt foi MODIFICADO (Alteração de Permissões)
  Informação inicial: USER=rubenmarques, GROUP=rubenmarques, SIZE=31, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=8b12e603872afe2543278d9d248834ebfde21239366f31d347f122c99cee5266
  Informação atual:  USER=rubenmarques, GROUP=rubenmarques, SIZE=31, PERM=777, MTIME=2026-11-05_14:52:18, SHA256=8b12e603872afe2543278d9d248834ebfde21239366f31d347f122c99cee5266
-------------------------------------------------------------------------------------------------------------------------------
/tmp/demo_vp1/ficheiro1.txt foi MODIFICADO (Alteração de Conteúdo)
  Informação inicial: USER=rubenmarques, GROUP=rubenmarques, SIZE=31, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=6e2d9af781d1156654bffbd1904a26e21f008a9ba348974f1ef14de6f1144257
  Informação atual:  USER=rubenmarques, GROUP=rubenmarques, SIZE=42, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=29886f0e0d9dd6c8867a2c16104a2b7d16efb9ddcae9a1ec87084a445bca7a20
-------------------------------------------------------------------------------------------------------------------------------
/tmp/demo_vp1/documentos/manual.txt foi APAGADO
 Informação inicial: USER=rubenmarques, GROUP=rubenmarques, SIZE=28, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=d85cf9342c24fdb57e41c2185716be1b7c865f1e428c4c51aed6b804dcf43e54
 Informação atual: [Ficheiro não encontrado no sistema]
-------------------------------------------------------------------------------------------------------------------------------
/tmp/demo_vp2/backup/dados.txt foi MODIFICADO (Alteração de Conteúdo)
  Informação inicial: USER=rubenmarques, GROUP=rubenmarques, SIZE=30, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=160bcfe05cc4ab0baa741f0136dde3792c09bdb73e6d5005798c8240f3ebcb1a
  Informação atual:  USER=rubenmarques, GROUP=rubenmarques, SIZE=49, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=b5c431bd467d430a3c8ceff3920a9727c89895d962a2ee2c4c2cc0423c1b150b
-------------------------------------------------------------------------------------------------------------------------------
/tmp/demo_vp1/ficheiro_novo.txt é NOVO
  Informação atual:  USER=rubenmarques, GROUP=rubenmarques, SIZE=29, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=6721f8459ff792b8ee2d86879ba8e10a7b46852031d046c849af739713d9af7d
-------------------------------------------------------------------------------------------------------------------------------
/tmp/demo_vp2/backup/novo_dir2.txt é NOVO
  Informação atual:  USER=rubenmarques, GROUP=rubenmarques, SIZE=22, PERM=664, MTIME=2026-11-05_14:52:18, SHA256=7cdf0127b067729c8f5f3118b2aa2eddecdce5e1c516944592fd6218d5101fa5
-------------------------------------------------------------------------------------------------------------------------------
Resumo da Verificação: 4 verificados | 3 Modificado | 1 Apagado | 2 Novo
```

Teste 3 – Registo no syslog (-s)

```
→ Comando para consultar os registos no Syslog: journalctl -t vp -n 7 --no-pager
nov 05 14:52:18 rubenmarques vp[10674]: /tmp/demo_vp1/ficheiro2.txt foi MODIFICADO (Alteração de Permissões)
nov 05 14:52:18 rubenmarques vp[10702]: /tmp/demo_vp1/ficheiro1.txt foi MODIFICADO (Alteração de Conteúdo)
nov 05 14:52:18 rubenmarques vp[10703]: /tmp/demo_vp1/documentos/manual.txt foi APAGADO
nov 05 14:52:18 rubenmarques vp[10731]: /tmp/demo_vp2/backup/dados.txt foi MODIFICADO (Alteração de Conteúdo)
nov 05 14:52:18 rubenmarques vp[10734]: /tmp/demo_vp1/ficheiro_novo.txt é NOVO
nov 05 14:52:18 rubenmarques vp[10759]: /tmp/demo_vp2/backup/novo_dir2.txt é NOVO
```

Teste 4 – Validação de erros e argumentos inválidos

Em todos os casos abaixo foi também apresentado o modo de uso do comando (msg_erro) apesar de ser omitido na tabela

Caso	Comando	Output
A	`./vp.sh -p /tmp/demo_vp1 -v`	Não é possível introduzir `-p` e `-v` simultaneamente.
B	`./vp.sh -x`	`Invalid switch -x`
C	`./vp.sh -f`	Falta argumento para `-f`
D	`./vp.sh -p`	Tem de ser indicado algum diretório.
E	`./vp.sh -v -f /tmp/bd_inexistente_999.pdb`	A base de dados `/tmp/bd_inexistente_999.pdb` não existe, tem de ser criada com a flag `-p`.
F	`./vp.sh -p /tmp/diretorio_que_nao_existe_123 -f /tmp/.pdb_demo`	O diretório `/tmp/diretorio_que_nao_existe_123` não existe. Base de dados de proteção `/tmp/.pdb_demo` criada com sucesso. Ficheiros processados: 0 | Diretórios protegidos: (vazio)

Análise dos Resultados

Proteção: Dos cinco ficheiros existentes, quatro foram registados: o `app.log` foi excluído pelo padrão `-e "*.log"`. A mensagem final indica os dois diretórios protegidos e a base de dados contém uma linha `#DIR|` para cada um, o que confirma que `-p` aceita vários diretórios.
O `documentos/manual.txt` e o `backup/dados.txt` estão em subpastas e foram registados, o que confirma a recursividade do `find`.

Deteção de alterações: Os quatro tipos de evento foram identificados corretamente, nos dois diretórios:

- Com o `chmod 777`, apenas o campo `PERM` mudou (`664` para `777`), e o `vp` descreveu a alteração só como "Permissões", distinguindo-a de uma alteração de conteúdo.
- Com o `echo >>`, mudaram `SIZE` (`31` para `42` no `ficheiro1.txt` e `30` para `49` no `dados.txt`), `SHA256` e `MTIME` (um segundo depois), e a descrição refletiu os dois tipos de alteração: "Conteúdo e Data de modificação".
- O ficheiro removido foi assinalado como apagado, mostrando a informação que tinha quando foi protegido.
- Os dois ficheiros novos foram assinalados como novos, um na raiz do primeiro diretório e outro numa subpasta do segundo, graças às linhas #DIR|, que indicam onde procurar.

Exclusão: O `novo.log` foi criado antes da verificação e não foi assinalado como novo, o que mostra que excluir também se aplica aos ficheiros lidos do disco. O resumo (4 verificados) corresponde aos quatro ficheiros registados na base de dados.

Registo no sistema: As seis mensagens da verificação aparecem no `journal` com a etiqueta `vp`, o que valida o `logger -p user.alert -t vp`.

Validação de argumentos: Todos os casos inválidos (A a F) foram rejeitados com uma mensagem e o modo de uso, sem executar nenhuma operação.

Conclusões

O comando `vp` cumpre os requisitos definidos: protege um ou mais diretórios de forma recursiva, guarda metadados e hash SHA-256 numa base de dados só de leitura e, na verificação, deteta ficheiros modificados, apagados e novos, identificando se a alteração foi de posse, permissões, conteúdo ou data de modificação. As opções `-f`, `-e` e `-s` funcionaram nos testes realizados, que incluíram a proteção de dois diretórios em simultâneo, e os erros de utilização são detetados e comunicados ao utilizador.

A divisão do script em funções (`obter_metadados()`, `calcular_perm()`, `excluir()`, `proteger()`, `verificar()` e `alerta()`) evitou código repetido, em particular na exclusão de ficheiros, usada nos dois modos. Os testes em cenários reais levaram a melhorias importantes, como a inclusão da data de modificação, o uso do `find` para garantir a recursividade e o registo dos diretórios protegidos na base de dados.

O trabalho permitiu consolidar o uso de `getopts`, `find`, `diff`, `grep`, `awk`, `sha256sum` e `logger`, bem como o tratamento de texto em Bash.

Referências

- Enunciado do Trabalho Prático #1 – Programação em Bash, Sistemas Operativos
  "Algumas pistas para a resolução deste trabalho:"
  - Agrupar funcionalidades em funções
  - Adotar estratégia de implementação incremental
  - Executar vários testes
  - Comentar código
  - Exemplos de interação
  - Vários comando recomendados
- Manuais (`man`) dos comandos `getopts`, `ls`, `find`, `diff`, `grep`, `awk`, `tr`, `sha256sum` e `logger`.
- Free Software Foundation, The GNU Bash Reference Manual
  https://www.gnu.org/software/bash/manual/html_node/index.html
- Ficheiro "Introdução à programação em Shell" fornecido no e-learning da cadeira (adaptado dos apontamentos de Dulce Domingos – FCUL)
- Site fornecido no guião da aula prático nº2
  https://mywiki.wooledge.org/BashGuide